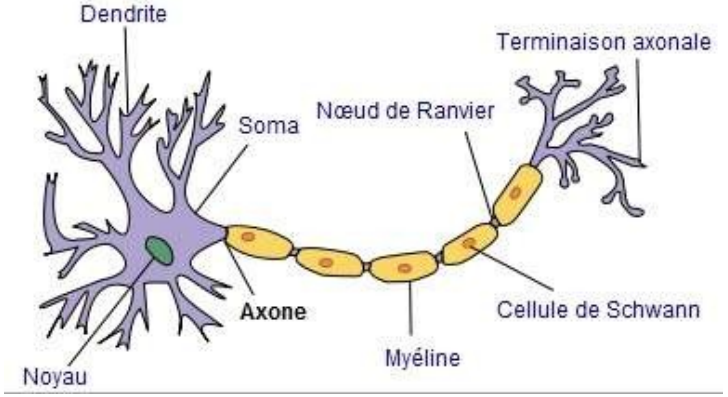
Artificial Intelligence for robotics

Part 3: the formal neuron

Simon Gay

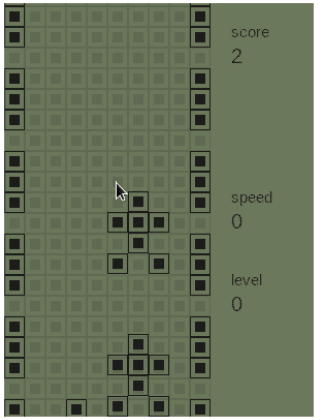
Artificial Intelligence for Robotics: the formal neuron

- **A simplified model of the neuron:**
  - biological neuron:
    - Dendrites collecting input signal
    - A body integrating input signals
    - An axon transmitting the signal
    - Activity defined by impulsion (frequency)



  - The model must be simplified to work on a computer
    - /!\ The formal neuron is an VERY simplified version of the biological neuron

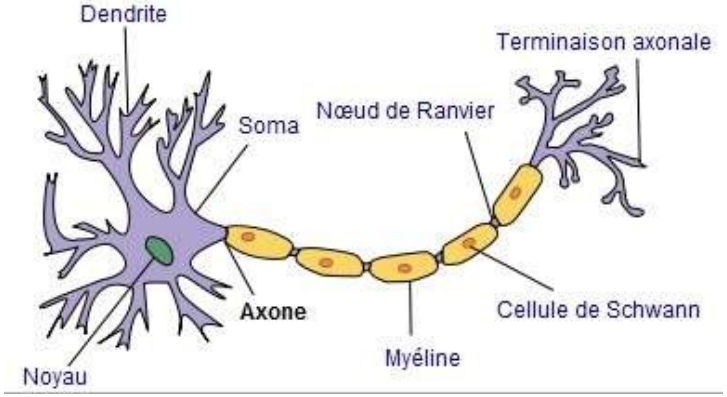




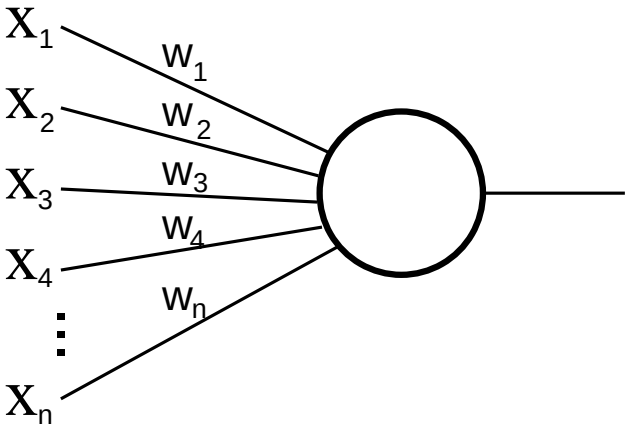
https://github.com/tshiamobhuda/Racing

Artificial Intelligence for Robotics: the formal neuron

- **A simplified model of the neuron:**
  - biological neuron:
    - Dendrites collecting input signal
    - A body integrating input signals
    - An axon transmitting the signal
    - Activity defined by impulsion (frequency)

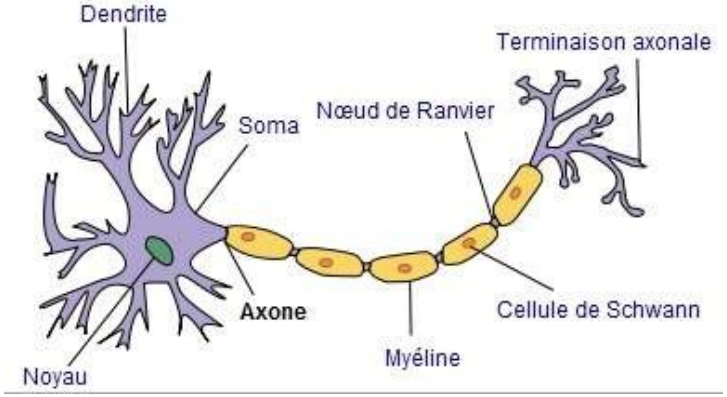


  - The model must be simplified to work on a computer
    - The frequency is replaced by a real value
    - Input: a vector of values
      - Vector X = { x_1, x_2, x_3, … , x_n }
    - A vector of reals for synapses
      - Vector W = { w_1, w_2, w_3, … , w_n }

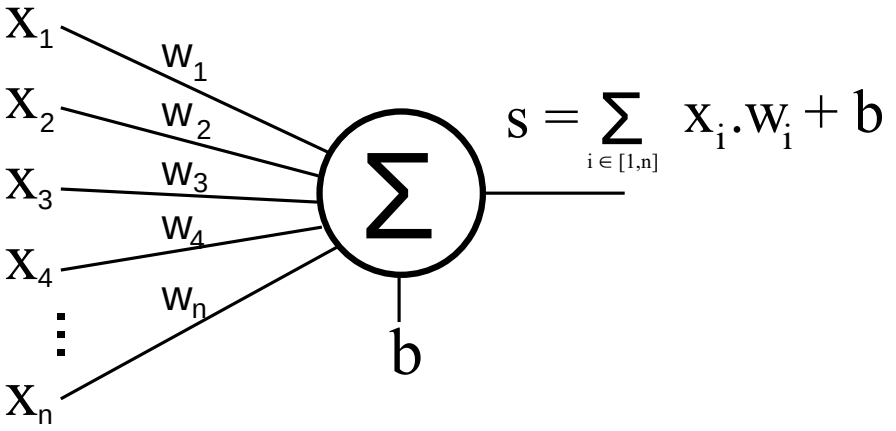


Artificial Intelligence for Robotics: the formal neuron

- **A simplified model of the neuron:**
  - biological neuron:
    - Dendrites collecting input signal
    - A body integrating input signals
    - An axon transmitting the signal
    - Activity defined by impulsion (frequency)

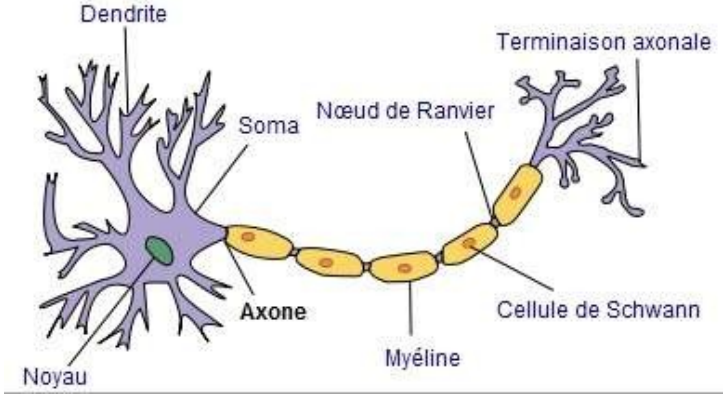


  - The model must be simplified to work on a computer
    - For integration of input values:
      - Sum of inputs values weighted by synaptic weights
      - A bias b

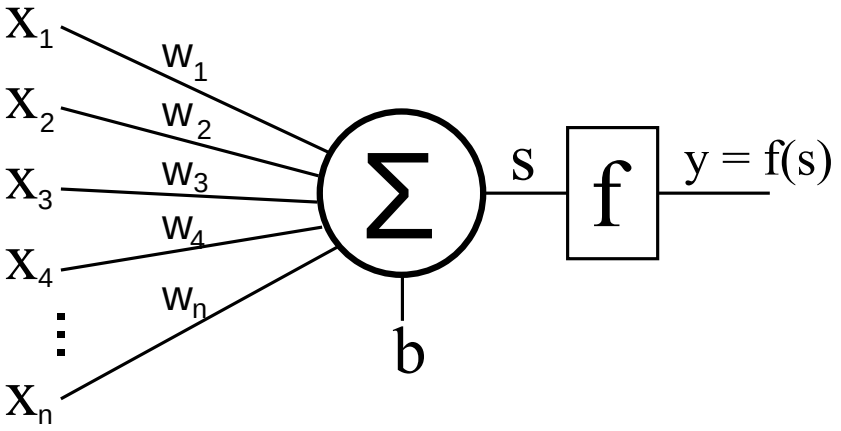


Artificial Intelligence for Robotics: the formal neuron

- **A simplified model of the neuron:**
  - biological neuron:
    - Dendrites collecting input signal
    - A body integrating input signals
    - An axon transmitting the signal
    - Activity defined by impulsion (frequency)



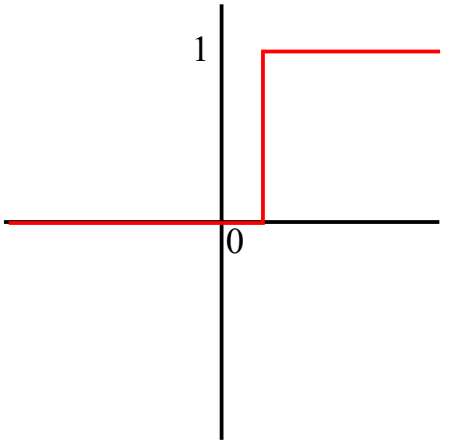
  - The model must be simplified to work on a computer
    - The output passes through an activation function
      - Allows bounding the output value



Artificial Intelligence for Robotics: the formal neuron

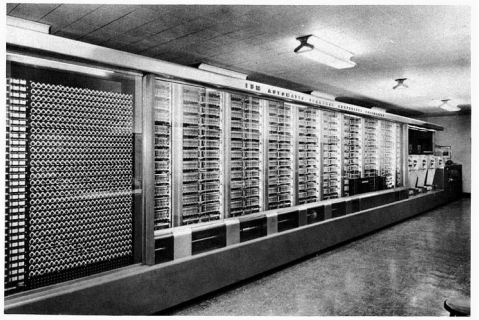
- **Which activation function ?**

  - Threshold function: S = 1 if y>threshold, S=0 else



    (note: the threshold value is not important because of the bias)

  - Function used by first formal neuron models (Perceptrons, proposed by Warren McCulloch and Walter Pitts in **1943**)

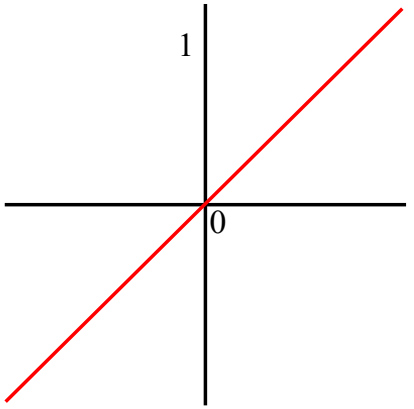

II Front View of the Calculator

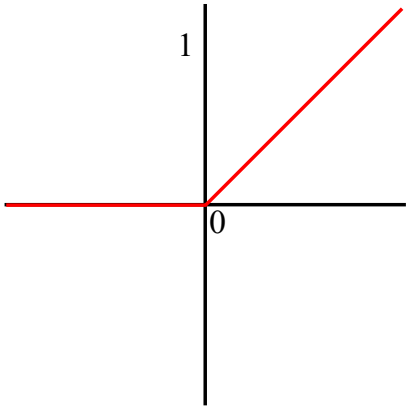
  - Some limitations:
    - We cannot compare output values of multiple neurons
    - The strength of the signal cannot be transmitted to other neurons.

Artificial Intelligence for Robotics: the formal neuron

- **Which activation function ?**

  - Linear function and ReLU (Rectified Linear Unit)
    - y = x
    - y = max(0,x)





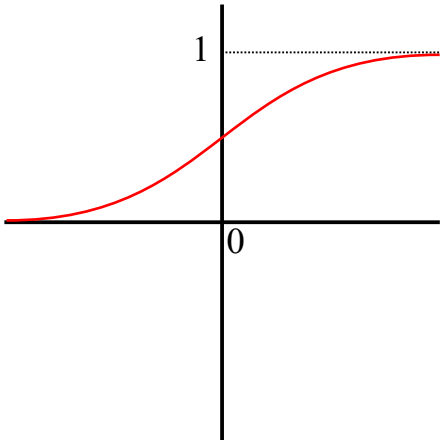
  - Very simple
  - ReLU remove negative values
    - Widely used in deep learning models

  - The output can be greater than 1
  - Gradient is constant

Artificial Intelligence for Robotics: the formal neuron

- **Which activation function ?**
  - Sigmoïd function
    - $y = \frac{1}{1 + e^{-x}}$

      (note: tanh is also used)



  - More complex to implement and compute
  - Great values generate a result close to 1 (or 0 if negative)
  - gradient tends to 0 for great values
    - Stabilization of weights (variations becomes increasingly small, but still present)
    - But gradient “loss” (a problem when using a large number of layers)
  - Another great advantage we will see later

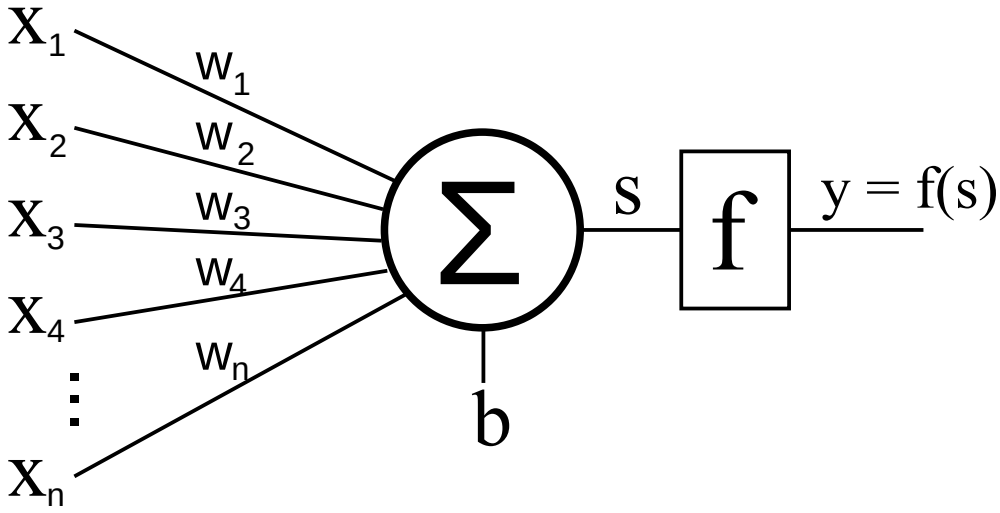
Artificial Intelligence for Robotics: the formal neuron

- **A simplified model of the neuron:**
  - The formal neuron
    - An input vector X
    - A weight vector W
    - An additional bias b
      - Equivalent to an input at 1

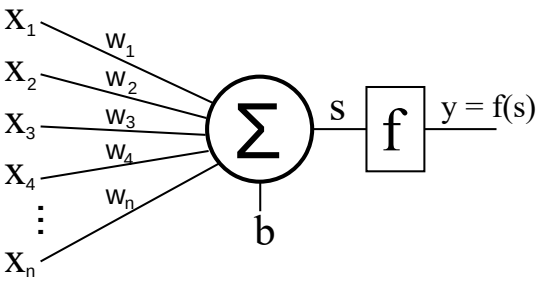
$$s = \sum_{i \in [1,n]} x_i . w_i$$

    - An activation function

      Y = f (S)



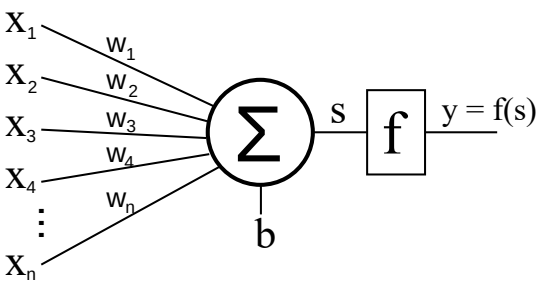
Artificial Intelligence for Robotics: the formal neuron



- **Supervised learning methods**
  - Hebb rule (1949) : "*cells that fire together, wire together*"
    - $w_i^{t+1} = w_i^t + \alpha \,.\, x_i \,.\, r$: if *r* and x_i have the same sign, w_i is reinforced

      α is the learning rate (limits weight variations)
  - Delta rule: changes each weight w_i proportionally to:
    - The input value (the higher the value of the input, the greater its impact on the result)
    - The difference between result and expected output (the greater the difference, the more the weights need to be adjusted)

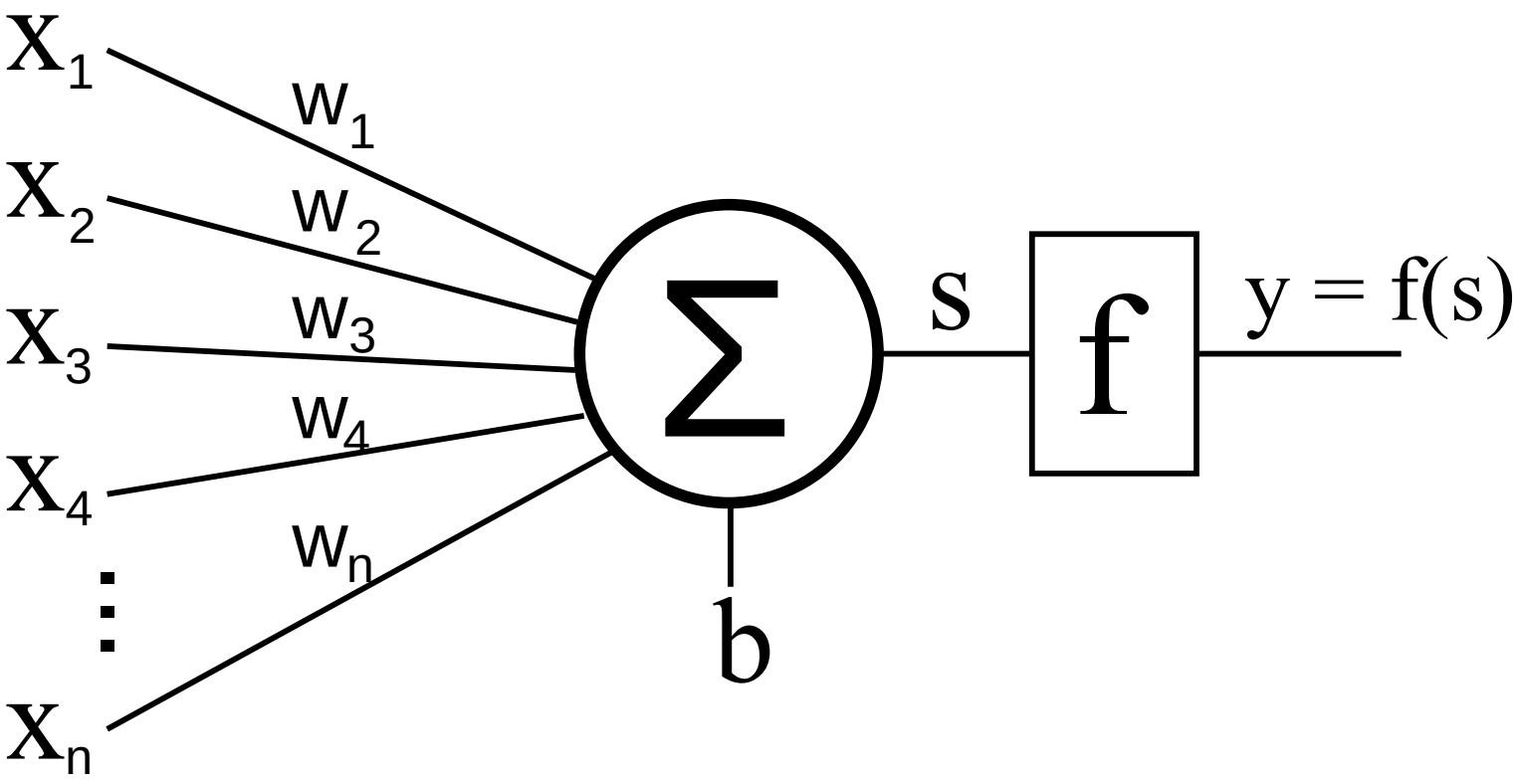
Artificial Intelligence for Robotics: the formal neuron



- **Supervised learning methods**
  - Delta rule: we compute the difference between expected result r and output y of the neuron:
    - $\Delta = r - y$
    - Perceptron (Rosenblatt, 1957): Treshold function, thus $\Delta \in \{ -1 ; 0 ; 1 \}$
    - Widrow-Hoff rule (1960): continuous activation function, thus $\Delta \in \mathbb{R}$
  - Each weight is updated:
    - Principles: weights are adjusted proportionally to:
      - Input value
      - Différence (Δ) between output and result
    - $w_i^{t+1} = w_i^t + \alpha \cdot x_i \cdot \Delta$

      α is the learning rate. Same update for the bias b
  - The learning process is performed on a (very) high number of examples

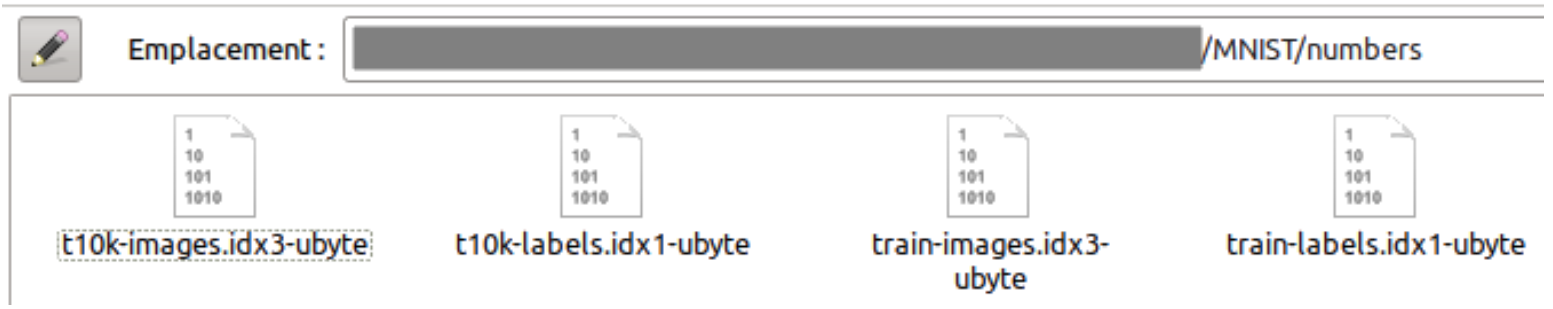
Artificial Intelligence for Robotics: the formal neuron



Let’s code !

Artificial Intelligence for Robotics: the formal neuron

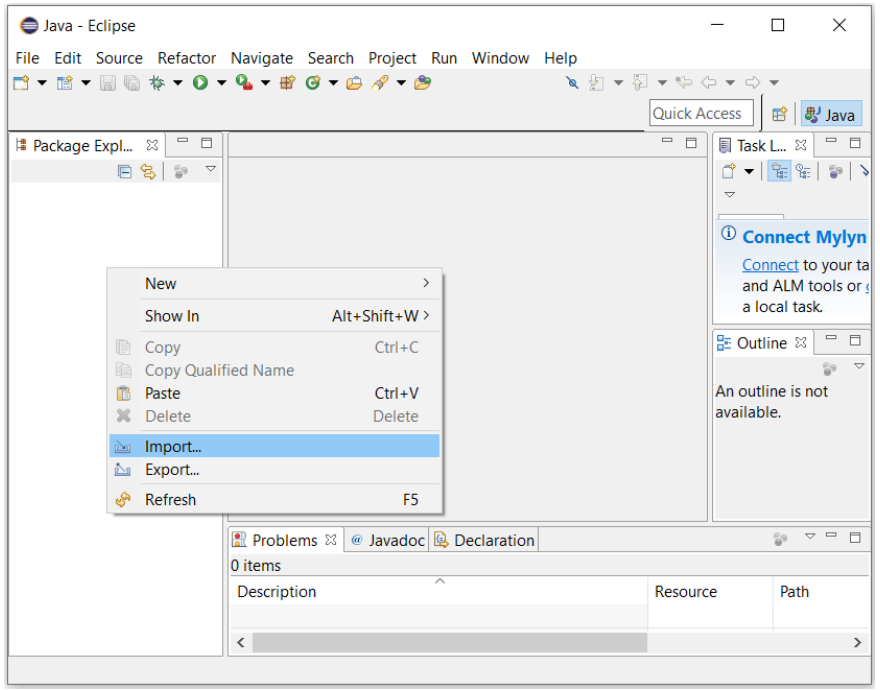
- **Environment**
  - Create a new folder ‘workspace_AI’ as our Java workspace
  - Download on ***https://github.com/gaysimon/AI_for_robotics*** the project archive **‘1_Neuron.zip’**
  - Download the four dataset files on ***https://github.com/fgnt/mnist*** (‘.gz’ files), **unzip** them and place them in a folder and subfolder “**MNIST/numbers/**” on your personal space

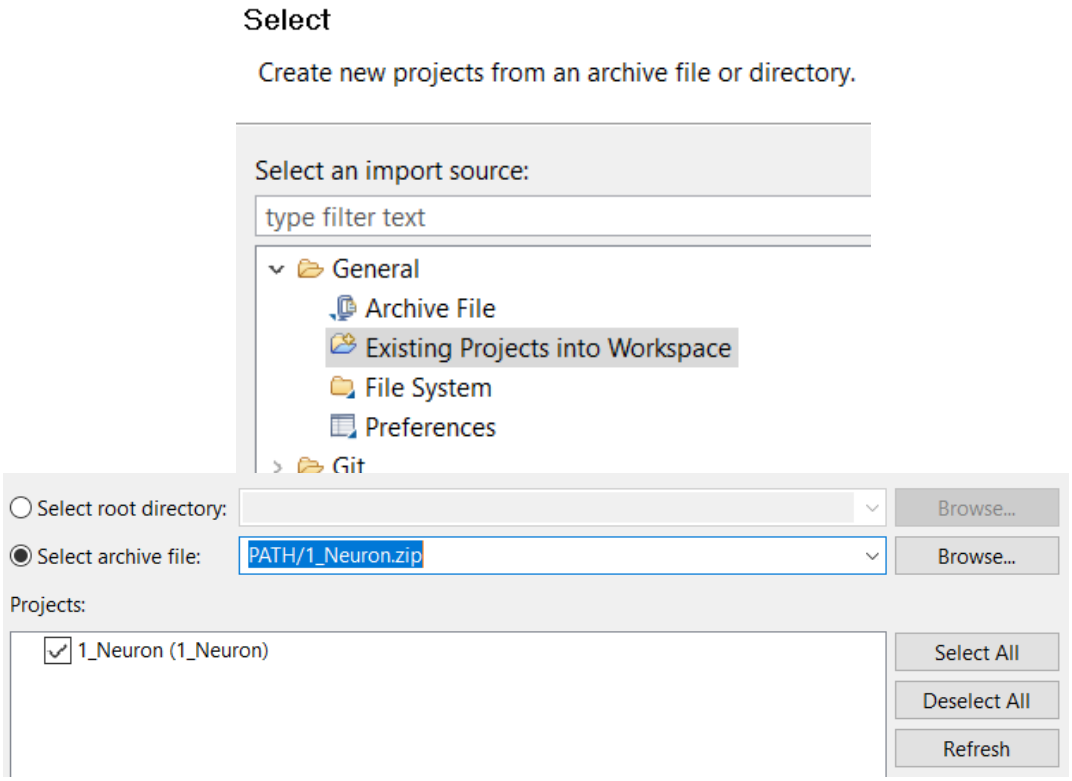


  - Open Eclipse and use your workspace folder
    - In the welcome panel, click on ‘**workbench**’ arrow

Artificial Intelligence for Robotics: the formal neuron

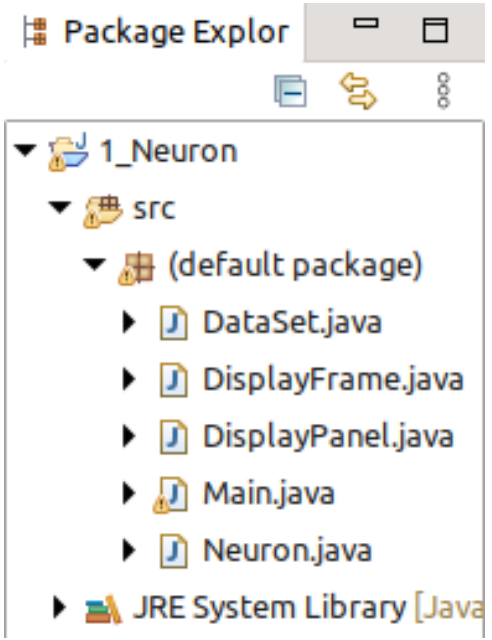
- **Environment**
  - In the Package Explorer, right-click and select “Import”
  - Select “General” then “Existing Projects into Workspace”, then click “Next”
  - Check “Select archive file”, select the archive “1_Neuron.zip” on your personal space, check the project “1_Neuron” and then click on “Finish”





Artificial Intelligence for Robotics: the formal neuron

- **Environment**
  - The project contains 5 classes:
    - **Main**: contains the main function, perform the training and evaluation process
    - **Neuron**: functions implementing a formal Neuron
    - **DataSet:** contains a function to load datasets
    - **DisplayFrame** and **DisplayPanel**: display panel

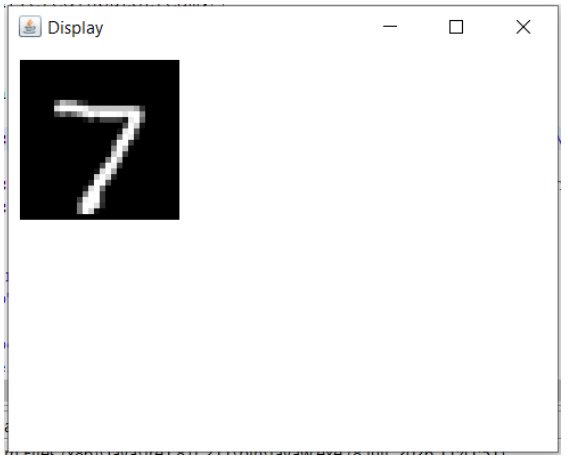


Artificial Intelligence for Robotics: the formal neuron

- **Environment**
  - In the **DataSet** class:
    - Define the path to the MNIST/numbers/ folder (do not forget the “/” at the end)
      - /!\ on Windows, use “\\” as folder separators

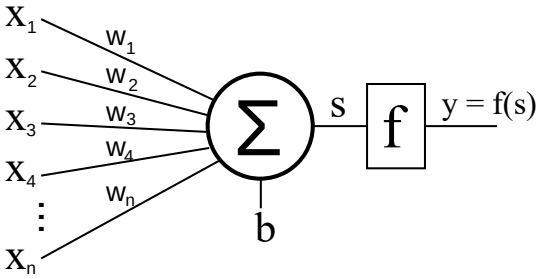
```
6  public class DataSet {
7
8      public static String PATH="Path_to_dataset/MNIST/numbers/"; // path to images
9
```

    - Start the project with button  . If everything is right, you should see the display panel



Artificial Intelligence for Robotics: the formal neuron

- **Implementing a formal neuron**



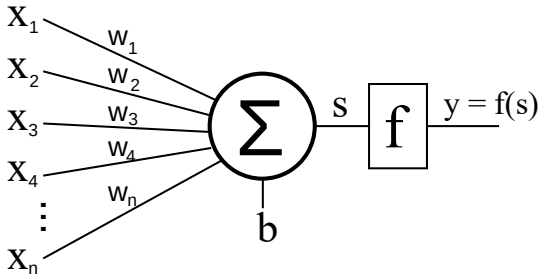
  - Lets add the data structures :
    - A vector of float for weights
    - A float for the bias
    - A float to store the output
    - A float for the delta (→ used for evaluation)

```
2  public class Neuron {
3
4      public float[] weights;
5      public float bias;
6      public float output=0;
7      public float delta=0;
8
9      public Neuron(){
```

  - Then, we define the constructor:
    - The only parameter is the number of weights
    - The weight vector can then be initialized

```
public Neuron(int size){
    weights=new float[size];
}
```

Artificial Intelligence for Robotics: the formal neuron



- **Implementing a formal neuron**
  - The output function **compute**

$$s = \sum_{i \in [1,n]} x_i . w_i + b$$

    - Weighted sum of inputs
    - Uses the input vector X as parameter
    - Return the result (float)

```
public float compute(float[] img){

    float sum=0;// weighted sum
    for (int i=0;i<img.length;i++){
        sum+= img[i] * weights[i] ;
    }
    sum+=bias;  // don't forget the bias

    output=activation(sum);

    return output;
}
```

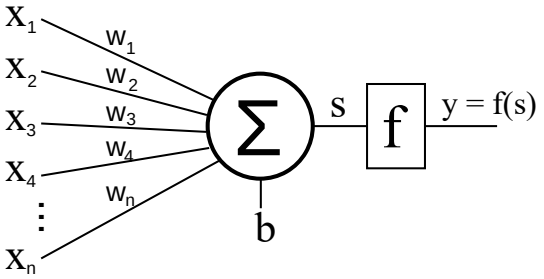
  - Then we add a sigmoid activation function:

$$y = \frac{1}{1 + e^{-x}}$$

    - Update the compute function !

```
public float activation(float x){
    return (float) ( 1 / (1+Math.exp(-x)));
}
```

Artificial Intelligence for Robotics: the formal neuron



- **Implementing a formal neuron**
  - The learning function **learn**
    - Delta rule:
      - $\Delta = r - y$
      - $w_i^{t+1} \leftarrow w_i^t + \alpha \cdot x_i \cdot \Delta$

```
2  public class Neuron {
3
4      public float learnRate=0.001f;
5
6      public float[] weights;
```

    - We add a learning rate set at 0.001
    - Function needs input vector AND expected result
    - Record the delta with the variable

```
public void learn(float[] img, int res){

    delta=res-output; // compute error

    for (int i=0;i<weights.length;i++){
        weights[i]+= learnRate * img[i] * delta;
    }
    bias+= learnRate * delta; // update bias
}
```

- **The neuron is complete !**

Artificial Intelligence for Robotics: the formal neuron

```java
public class Neuron {

    public float learnRate=0.001f;

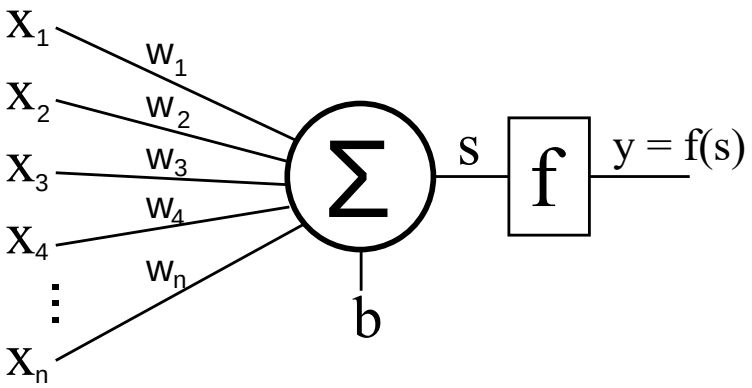
    public float[] weights;
    public float bias;
    public float output=0;
    public float delta=0;

    public Neuron(int size){
        weights=new float[size];
    }

    public float compute(float[] img){
        float sum=0;// weighted sum
        for (int i=0;i<img.length;i++){
            sum+= img[i] * weights[i] ;
        }
        sum+=bias;  // don't forget the bias
        output=activation(sum);
        return output;
    }

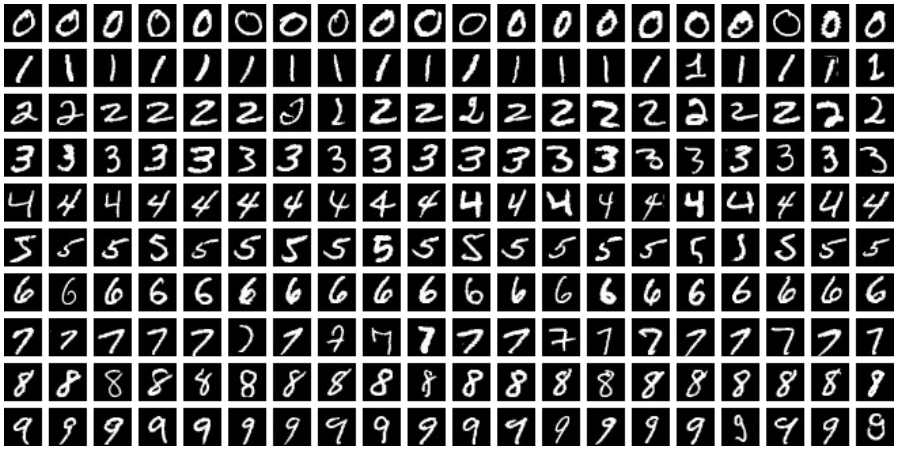
    public void learn(float[] img, int res){
        delta=res-output; // compute error
        for (int i=0;i<weights.length;i++){
            weights[i]+= learnRate * img[i] * delta;
        }
        bias+= learnRate * delta; // update bias
    }

    // activation function Sigmoid
    public float activation(float x){
        return (float) ( 1 / (1+Math.exp(-x)));
    }
}
```



Artificial Intelligence for Robotics: the formal neuron

- **Training process**
  - The training process is performed by the Main class.
  - We will use a famous dataset: MNIST
    - *Modified National Institute of Standards and Technology*
    - Created in 1994, widely used since 1998 for benchmarks
    - Contains 60 000 hand-written labeled digits for training and 10 000 for tests
    - Images of 28x28 pixels
    - The dataset files encode images as a unique sequence of *int* values (0-255)



Artificial Intelligence for Robotics: the formal neuron

- **Training process**
  - The function **DataSet.setData()** (provided) read a dataset and its labels and store them in matrices:
    - float[][] matrixImages: each line is a vector of 28x28 float (0-1) values encoding one image
    - int[] matrixLabels: each value is a label indicating the digit in each image (0 to 9)
  - The training process works as follow:
    - Load training dataset
    - For each dataset input:
      - Define the expected output of the neuron (0 or 1)
      - Compute the neuron’s result
      - Update weights

Artificial Intelligence for Robotics: the formal neuron

- **Training process**
  - First, set the size of the neuron
    - Vector of 28x28 values

```
33      //////////////
34      public Main(){
35
36          // initialize structures
37          neuron=new Neuron(size_x*size_y);
```

  - Then, we will create the training loop:
    - We use the global variable test (linked to the display panel)

```
// initialize display frame
display=new DisplayFrame(this);


for (test=0;test<matrixImages.length;test++){


    // TODO : learning


    //try {Thread.sleep(10);
    //} catch (InterruptedException e) {e.printStackTrace();}
}


System.out.println("learning completed");
```

Artificial Intelligence for Robotics: the formal neuron

- **Training process**
  - A neuron has an output between 0 and 1: our neuron can thus recognize only one digit. In global variables, select a digit to be recognized (0 to 9)

```
public static int number=3;          // selected digit for test
```

  - The expected output can be defined: if the label is equal to *number*, the output should be 1, otherwise, it should be 0.
  - Then, we can use function compute to get the output and learn to train the neuron

```
for (test=0;test<matrixImages.length;test++){

    // define expected result
    int res=0;
    if (matrixLabels[test]==number) res=1;

    // process neuron
    neuron.compute(matrixImages[test]);
    neuron.learn(matrixImages[test], res);

    display.repaint(); // update display panel
}
```

Artificial Intelligence for Robotics: the formal neuron

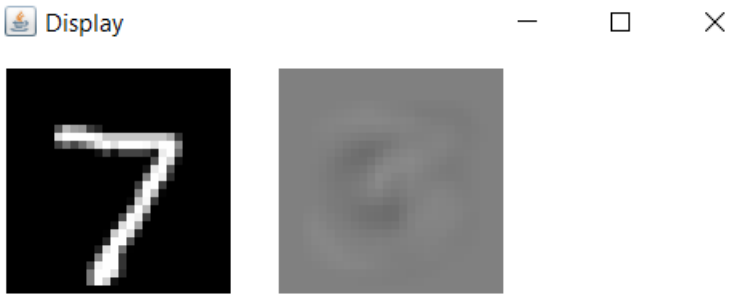
- **Training process**
  - You can uncomment the neuron weights display in the DisplayPanel class

```
for (int i=0;i<Main.size_x;i++){
    for (int j=0;j<Main.size_y;j++){
        val=(int)(main.neuron.weights[i+Main.size_x*j]*50)+128;
        if (val<0) val=0;
        if (val>255) val=255;
        g.setColor(new Color(val,val,val));
        g.fillRect(180+5*i, 10+5*j, 5, 5);
    }
}
```

  - You can also uncomment the delay in the loop to observe digit images

```
    display.repaint(); // update display panel

    try {Thread.sleep(10);
    } catch (InterruptedException e) {e.printStackTrace();}
}
```



Artificial Intelligence for Robotics: the formal neuron

- **Training process**
  - Processing the dataset once is often not sufficient. We will process it multiple times. Each dataset training is called an **epoch**
  - We add a loop to perform 20 epochs

```
// initialize display frame
display=new DisplayFrame(this);

// epoch loop
for (epoch=0;epoch<20;epoch++){

    // dataset loop
    for (test=0;test<matrixImages.length;test++){

        // define expected result
        int res=0;
        if (matrixLabels[test]==number) res=1;

        // process neuron
        neuron.compute(matrixImages[test]);
        neuron.learn(matrixImages[test], res);

        display.repaint(); // update display panel

        //try {Thread.sleep(10);
        //} catch (InterruptedException e) {e.printStackTrace();}
    }

}
```

Artificial Intelligence for Robotics: the formal neuron

- **Training process**
  - We then measure performances by collecting all delta values of an epoch

```
// epoch loop
for (epoch=0;epoch<20;epoch++){

    float sumdelta=0; // initialize the sum of errors

    // dataset loop
    for (test=0;test<matrixImages.length;test++){
        // define expected result
        int res=0;
        if (matrixLabels[test]==number) res=1;
        // process neuron
        neuron.compute(matrixImages[test]);
        neuron.learn(matrixImages[test], res);

        sumdelta+=Math.abs(neuron.delta); // add error (absolute value)

        display.repaint(); // update display panel
        //try {Thread.sleep(10);
        //} catch (InterruptedException e) {e.printStackTrace();}
    }

    // display sum of errors
    System.out.println("epoch n°"+epoch+" : "+(sumdelta/matrixImages.length));
}
```

Artificial Intelligence for Robotics: the formal neuron

- **Training process**
  - We then measure performances by collecting all delta values of an epoch

```
epoch n°0 : 0.08726755
epoch n°1 : 0.060714222
epoch n°2 : 0.055588827
epoch n°3 : 0.05288086
epoch n°4 : 0.051110104
epoch n°5 : 0.04982587
epoch n°6 : 0.04883459
epoch n°7 : 0.048038695
epoch n°8 : 0.047379997
epoch n°9 : 0.04682368
epoch n°10 : 0.046345204
epoch n°11 : 0.04592855
epoch n°12 : 0.04556147
epoch n°13 : 0.04523546
epoch n°14 : 0.044943605
epoch n°15 : 0.04468028
epoch n°16 : 0.044441562
epoch n°17 : 0.044223938
epoch n°18 : 0.044024635
epoch n°19 : 0.043841813
```



Artificial Intelligence for Robotics: the formal neuron

- **Evaluation**
  - The evaluation must be performed on different inputs than for training
  - MNIST proposes a second dataset of 10 000 images

  - After the training loop, add another dataset loop (no epoch), with variable *test*
  - In this loop, we only compute the neuron output and display the result

```
// load evaluation dataset
setData("t10k-images.idx3-ubyte","t10k-labels.idx1-ubyte");

// TODO : evaluation
for (test=0;test<matrixImages.length;test++){

    System.out.print(matrixLabels[test]+" -> ");     // display label
    float res=neuron.compute(matrixImages[test]);    // compute output
    if (res<0.001) res=0;   // avoid exponential number display
    System.out.println(res);                         // display result

}
```

Artificial Intelligence for Robotics: the formal neuron

- **Evaluation**
  - We define an error as follow
    - Label is the right number but prediction is <0.5 (false negative)
    - Label is not the right number but prediction is >0.5 (false positive)
  - At the end, we display the number of errors (for 10 000 tests)

```
//////////////////////////////////////////////////////
// evaluation of the neuron
int errors=0;

// load evaluation dataset
setData("t10k-images.idx3-ubyte","t10k-labels.idx1-ubyte");

// TODO : evaluation
for (test=0;test<matrixImages.length;test++){

    System.out.print(matrixLabels[test]+" -> ");    // display label
    float res=neuron.compute(matrixImages[test]);   // compute output
    if (res<0.001) res=0;   // avoid exponential number display
    System.out.println(res);                        // display result

    // count errors
    if ( (matrixLabels[test]==number && res<0.5) || (matrixLabels[test]!=number && res>0.5) ) errors++;
}

// display number of errors
System.out.println("total "+errors+" errors");

display.repaint();
```

Artificial Intelligence for Robotics: the formal neuron

- **Evaluation**
  - You can try to increase the number of epochs
    - The error decreases, but it seems to be stagnating: increasing the number of epoch will never leads to an error of 0
    - Sometimes, the number of errors increases: the neuron becomes too specialized with training dataset and cannot generalize
      - Overfitting !
  - **Your neuron is working !**